



FACULTY OF ENGINEERING AND NATURAL SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

PROGRAMMING LANGUAGE PROJECT: ISA

2025-2026 SPRING

STUDENT ID	NAME/SURNAME
230315020	Ayşe ARSLAN
230315038	Zeynep İclal BİLGİLİ
230315074	Sude Naz AY

TABLE OF CONTENTS

1. INTRODUCTION
2. BASE TEXTBOOK CODE USED
3. OVERVIEW OF THE DESIGNED LANGUAGE
4. RESERVED KEYWORDS AND TOKENS
5. BNF / EBNF GRAMMAR
6. LEXICAL ANALYZER DESIGN
7. SYNTAX ANALYZER DESIGN
8. STATE DIAGRAMS
9. SYMBOL TABLE / LOOKUP TABLE
10. SAMPLE PROGRAMS AND TEST RESULTS
11. CONCLUSION

1. INTRODUCTION

This report presents the design and implementation process of **ISA**, a Turkish-based mini programming language developed as a Programming Languages course project. The language was created to reduce the language barrier encountered by Turkish-speaking beginner programmers and to provide a more intuitive learning environment through Turkish syntax and keywords.

The architecture of ISA is based on the lexical analysis and recursive-descent parsing techniques presented in Robert W. Sebesta's *Concepts of Programming Languages* textbook. While the original textbook examples focus mainly on arithmetic expression parsing, the ISA project significantly extends these concepts into a complete mini programming language supporting variable declarations, arithmetic expressions, conditional statements, loops, output operations, symbol table management, and syntax error reporting.

The compiler front-end of ISA consists of two major components: a Lexical Analyzer (Lexer) and a Syntax Analyzer (Parser). The lexical analyzer reads source code character by character and converts lexemes into tokens, while the syntax analyzer verifies whether the token sequence conforms to the grammar rules defined for the language.

A major objective of the project was to create a language that naturally supports Turkish users. Therefore, Turkish reserved keywords such as "sayı", "ise", "değilse", "iken", and "yazdır" were introduced. In addition, the compiler supports Turkish characters (ç, ğ, ı, ö, ş, ü) within both identifiers and reserved keywords by utilizing Java's Unicode support and custom character classification methods.

Several improvements were implemented beyond the textbook examples. These include support for line comments (*//*), line-number-aware error reporting, symbol table validation, variable declaration with initialization, arithmetic expression parsing with precedence rules, and semantic checks for undeclared variables.

The project demonstrates how theoretical concepts such as lexical analysis, finite-state token recognition, recursive-descent parsing, and symbol table management can be transformed into a practical compiler front-end implementation using Java.

2. BASE TEXTBOOK CODE USED

The ISA programming language was developed by using several lexical analysis and recursive-descent parsing examples presented in Robert W. Sebesta's *Concepts of Programming Languages* textbook. The textbook examples provided the fundamental compiler architecture and parsing methodology used throughout the project. However, the original examples were significantly extended and modified to support the ISA language's Turkish syntax, symbol table management, and improved error handling mechanisms.

2.1 Which Textbook Examples Were Used

The following concepts and example structures from the textbook were used as the foundation of the project:

- Lexical Analyzer (Scanner) architecture
- Character classification using LETTER, DIGIT, UNKNOWN, and EOF classes
- Character reading mechanism through the `getChar()` function

- Lexeme construction using the addChar() function
- Whitespace elimination using the getNonBlank() function
- Token generation using the lex() function
- Recursive-Descent Parsing strategy
- Arithmetic expression parsing using expr(), term(), and factor() functions
- Token-based syntax validation methodology

These examples provided the basic framework for implementing both the lexical analyzer and syntax analyzer of the ISA language.

2.2 What We Kept

Several components from the textbook examples were preserved without major structural changes:

- The overall Lexer → Parser workflow
- Global lexical analyzer variables such as charClass, nextChar, nextToken, lexeme, and lexLen
- Recursive-descent parser architecture
- Expression parsing hierarchy consisting of expr(), term(), and factor()
- Token-based grammar validation approach
- Character-by-character source code scanning process

The original compiler design philosophy presented in the textbook was maintained throughout the implementation.

2.3 What We Modified

To support the ISA language and Turkish-based syntax, several parts of the original textbook implementation were modified:

- English keywords were replaced with Turkish keywords.

Textbook Style	ISA Style
int	sayi
if	ise
else	degilse
while	iken
print	yazdir

- Identifier recognition was extended to support Turkish characters such as ç, ğ, ı, ö, ş, and ü.
- The lexical analyzer was modified to recognize multi-character relational operators such as ==, !=, <=, and >=.
- Grammar rules were redesigned according to the ISA language structure.

- Syntax and lexical error messages were customized to provide more descriptive feedback.
- The print statement was extended to accept complete arithmetic expressions instead of only identifiers or literals.

2.4 What We Added

Several original features were implemented that do not exist in the textbook examples:

- Symbol Table implementation using a HashSet data structure
- Detection of undeclared variable usage during parsing
- Line number tracking for error reporting
- Support for variable initialization during declaration

Example:

```
sayi x = 10;
```

- Support for line comments beginning with //

Example:

```
// This is a comment
```

- Turkish keyword support
- Full arithmetic expression support inside the yazdir() statement

Example:

```
yazdir(x + y * 2);
```

- Enhanced syntax error reporting displaying:
 - Line number
 - Current token
 - Current lexeme
 - Detailed error description

These additions transformed the textbook examples into a complete compiler front-end capable of performing lexical analysis, syntax analysis, symbol table validation, and detailed error reporting for the ISA programming language.

3. OVERVIEW OF THE DESIGNED LANGUAGE

ISA is a Turkish-based imperative mini programming language designed to teach fundamental programming concepts — variable management, arithmetic computation, and control flow — using a syntax tailored to the Turkish language. To maintain a focus on core algorithmic principles rather than complex data structures, the language currently supports only the integer (sayi) data type.

3.1 Design Goals

ISA was built around four core principles: reducing the foreign-language barrier for Turkish-speaking beginner programmers, providing an intuitive syntax that mirrors native Turkish logical reasoning, demonstrating core compiler construction concepts such as lexical analysis, parsing, and symbol table management, and supporting the full Unicode Turkish character set in both keywords and user-defined identifiers.

3.2 Supported Features

Statement and Block Management: Every statement in ISA must be explicitly terminated with a semicolon (;). Code blocks — such as those belonging to conditionals or loops — are scoped using curly braces ({ }).

Variable Declaration: Integer variables are declared using the `sayi` keyword. Initialization at declaration time is also supported, so both `sayi x;` and `sayi x = 10;` are valid forms.

Arithmetic and Relational Operations: The language fully supports the four basic arithmetic operators (+, -, *, /) with correct mathematical precedence enforced through parentheses. For logical evaluations, all six standard relational operators are available: <, >, <=, >=, ==, and !=.

Conditional Branching: Program execution paths are controlled by `ise` (if) blocks with an optional `degilse` (else) fallback. For example:

```
ise (x > 5) {  
    yazdir(x);  
}  
degilse {  
    yazdir(0);  
}
```

Loop Structures: Iterative execution is handled by the `iken` (while) construct, which repeatedly executes its body block as long as the specified condition holds true:

```
iken (x < 10) {  
    x = x + 1;  
}
```

Output: The built-in `yazdir()` function outputs computed values or arithmetic expressions to the console. It accepts full arithmetic expressions as its argument, not just single variables — for example, `yazdir(x + y * 2);`.

Comment Support: Single-line comments are supported using the `//` prefix. The lexer discards everything from `//` to the end of the line, allowing programmers to annotate their code freely.

3.3 Compilation Platform

ISA is implemented entirely in Java, leveraging Java's native Unicode capabilities to seamlessly handle Turkish characters throughout both the lexer and parser. The source file is compiled with `javac ISA.java` and a program is run as `java ISA <source_file.isa>`.

4. RESERVED KEYWORDS AND TOKENS

The Lexical Analyzer of the ISA programming language converts source code into a sequence of tokens that can be processed by the Syntax Analyzer. During this phase, each lexeme is classified according to its role within the language. Reserved keywords represent predefined language constructs, while identifiers, literals, operators, and delimiters form the remaining lexical components of the language.

Keyword recognition is performed through the `kelimeTurunuBul()` method. If a scanned lexeme matches one of the predefined ISA keywords, the corresponding token class is generated; otherwise, the lexeme is classified as an identifier. The following table summarizes all token categories supported by the ISA programming language.

Keyword / Symbol (Lexeme)	Token Class (Token Name)	Description
sayi	SAYI_CODE	Declares a new integer variable
ise	ISE_CODE	Initiates a conditional if block
degilse	DEGILSE_CODE	Initiates an alternative else block
iken	IKEN_CODE	Initiates a while loop structure
yazdir	YAZDIR_CODE	Executes the built-in print operation
Letter sequence	IDENT_CODE	User-defined variable identifiers
Digit sequence	INT_LIT_CODE	Integer literal values
+	ADD_OP	Addition operator
-	SUB_OP	Subtraction operator
*	MULT_OP	Multiplication operator
/	DIV_OP	Division operator
=	ASSIGN_OP	Assignment operator
==	EQUAL_OP	Equality operator
!=	NOT_EQUAL_OP	Not-equal operator
<	LESS_OP	Less-than operator
>	GREATER_OP	Greater-than operator
<=	LESS_EQUAL_OP	Less-than-or-equal operator
>=	GREATER_EQUAL_OP	Greater-than-or-equal operator
;	SEMICOLON	Statement terminator
{	LEFT_BRACE	Opening block delimiter
}	RIGHT_BRACE	Closing block delimiter
(	LEFT_PAREN	Opening parenthesis
)	RIGHT_PAREN	Closing parenthesis
EOF	EOF_CODE	End-of-file marker

The token classes listed above constitute the lexical foundation of the ISA language. After tokenization, these tokens are passed to the Recursive-Descent Parser, which validates whether the source code conforms to the grammar rules defined for the ISA language.

5. GRAMMAR OF LANGUAGE – IN BNF FORMAT

`<program> ::= <ifade_listesi> <EOF>`

`<ifade_listesi> ::= { <ifade> }`

`<ifade> ::= <tanimlama>
 | <atama>
 | <kosul_ifadesi>
 | <dongu_ifadesi>
 | <yazdir_ifadesi>`

`<tanimlama> ::= "sayi" <identifier>`

```

[ "=" <expr> ] ";"

<atama> ::= <identifier> "=" <expr> ";"

<yazdir_ifadesi> ::= "yazdir" "(" <expr> ")" ";"

<kosul_ifadesi> ::= "ise"
    "(" <bool_expr> ")"
    "{"
        <ifade_listesi>
    "}"
    [ "degilse"
        "{"
            <ifade_listesi>
        "}"
    ]

<dongu_ifadesi> ::= "iken"
    "(" <bool_expr> ")"
    "{"
        <ifade_listesi>
    "}"

<bool_expr> ::= <expr>
    <relational_operator>
    <expr>

<expr> ::= <term>
    { ("+" | "-") <term> }

<term> ::= <factor>
    { ("*" | "/") <factor> }

<factor> ::= <identifier>
    | <integer_literal>
    | "(" <expr> ")"

<relational_operator> ::= "=="
    | "!="
    | "<"
    | ">"
    | "<="
    | ">="

<identifier> ::= TOKEN_IDENTIFIER

<integer_literal> ::= TOKEN_INTEGER_LITERAL

<EOF> ::= TOKEN_EOF

```

6. LEXICAL ANALYZER DESIGN

The Lexical Analyzer (Scanner) part of the compiler is responsible for reading characters from the source code and converting them into meaningful tokens. In this project, the lexical analyzer performs character reading, whitespace removal, comment handling, keyword recognition, and token generation operations.

The Lexical Analyzer structure consists of the following main functions:

- getChar()
- isTurkishLetter()
- addChar()
- getNonBlank()
- consumePending()
- kelimeTurunuBul()
- lex()

6.1 getChar() Function

This function reads the next character from the source file. After reading the character, its class is determined as a letter, digit, or unknown character.

Additionally, Turkish character support has been implemented. Therefore, Turkish variable names can also be used in the language.

The function performs the following operations:

- Line number tracking
- Letter checking
- Digit checking
- EOF checking

```

130  static void getChar() {
131      try {
132          int c = reader.read();
133          if (c != -1) {
134              nextChar = (char) c;
135              if (nextChar == '\n') satirNo++;
136
137              if (isTurkishLetter(nextChar)) {
138                  charClass = LETTER;
139              } else if (Character.isDigit(nextChar)) {
140                  charClass = DIGIT;
141              } else {
142                  charClass = UNKNOWN;
143              }
144          } else {
145              charClass = EOF_CLASS;
146          }
147      } catch (IOException e) {
148          charClass = EOF_CLASS;
149      }
150  }

```

6.2 isTurkishLetter() Function

This helper function allows Turkish characters to be recognized as letters.

Supported Turkish characters:

- ç, Ç
- ğ, Ğ
- ı, İ
- ö, Ö
- ş, Ş
- ü, Ü

This feature makes the language more user-friendly.

```
152     /** Türkçe dahil harfleri tanıyan yardımcı metod */
153     static boolean isTurkishLetter(char c) {
154         return Character.isLetter(c)
155             || c == 'ç' || c == 'Ç'
156             || c == 'ğ' || c == 'Ğ'
157             || c == 'ı' || c == 'İ'
158             || c == 'ö' || c == 'Ö'
159             || c == 'ş' || c == 'Ş'
160             || c == 'ü' || c == 'Ü';
161     }
```

6.3 addChar() Function

This function is used to append the currently read character into the lexeme array.

Lexeme length control is implemented to prevent overflow errors.

Responsibilities of this function:

- Add character into lexeme array
- Terminate lexeme properly
- Check maximum lexeme length

```
163     /**
164      * addChar() - Kitaptaki addChar() fonksiyonu
165      * nextChar'ı lexeme dizisine ekler.
166      */
167     static void addChar() {
168         if (lexLen <= 98) {
169             lexeme[lexLen++] = nextChar;
170             lexeme[lexLen] = 0;
171         } else {
172             System.out.println("Hata: Lexeme çok uzun (satır " + satirNo + ").");
173         }
174     }
175
176     /**
177      * getNonBlank() - Kitaptaki getNonBlank() fonksiyonu
178      * Boşlukları atlar VE '//' ile başlayan yorum satırlarını temizler.
179      */
```

6.4 getNonBlank() Function

This function removes whitespace characters and comment lines before token generation.

The following elements are skipped automatically:

- Spaces
- Tabs
- New lines
- // comment lines

Thanks to this structure, the parser only processes meaningful tokens.

The support for // style comments is one of the major improvements added to this project.

```

177 * getNonBlank() - Kitaptaki getNonBlank() fonksiyonu
178 * Boşlukları atlar VE '/' ile başlayan yorum satırlarını temizler.
179 */
180 static void getNonBlank() {
181     // Boşluk veya yorum satırı olduğu sürece döngüde kal
182     while (true) {
183         // Boşlukları atla
184         while (charClass != EOF_CLASS && Character.isWhitespace(nextChar)) {
185             getChar();
186         }
187         // Yorum satırı kontrolü - '/' ise satır sonuna kadar atla
188         if (charClass == UNKNOWN && nextChar == '/') {
189             // Bir sonraki karaktere ihtiyaç var; onu peek etmek için okuyoruz.
190             // ama geri alamayız, bu yüzden geçici olarak saklıyoruz.
191             // Yorum değilse DIV_OP için nextChar'ı '/' olarak bırakmak yeterli.
192             // Bu yüzden reader.mark/reset kullanmak yerine bir bayrak tutuyoruz:
193             // Eğer sonraki char yine '/' ise yorum, değilse dur.
194             char savedChar = nextChar;
195             int savedClass = charClass;
196             getChar(); // ikinci karakteri oku
197             if (nextChar == '/') {
198                 // Yorum satırı: satır sonuna kadar atla
199                 while (charClass != EOF_CLASS && nextChar != '\n') {
200                     getChar();
201                 }
202                 // Döngünün başına dön; sonraki satırdaki boşlukları da atla
203             } else {
204                 // Gerçek bir '/' operatörü - okuduğumuz ikinci char'ı "geri koy"
205                 // Bunu yapamayız doğrudan, bu yüzden nextChar/charClass'ı
206                 // geri yükleyip döngüden çıkıyoruz.
207                 // Çözüm: savedChar ve savedClass'ı geri yükle, ikinci char'ı
208                 // lookahead olarak tut (lex() içindeki UNKNOWN case'i bunu okur).
209                 // Ancak bu iki karakterli durumu lex() içinde çözmek daha temiz;
210                 // bu yüzden burada nextChar'ı sıfırlayıp işi lex()'e bırakıyoruz.
211                 // Mevcut nextChar ikinci char olarak lex()'in UNKNOWN dalında kullanılır.
212                 // '/' tokenını işaretlemek için önce nextChar'ı savedChar yap:
213                 // ** Bu yaklaşım çift-karakter lookahead gerektiriyor.
214                 // En temiz çözüm: ikinci karakteri pendingChar'a kaydetmek. **
215                 pendingChar = nextChar;
216                 pendingCharClass = charClass;
217                 hasPending = true;
218                 nextChar = savedChar;
219                 charClass = savedClass;
220                 return; // Döngüden çık; lex() '/' i işleyecek
221             }
222         } else {
223             break; // Ne boşluk ne yorum; gerçek bir token başlıyor
224         }
225     }
226 }
227
228 // — Tek karakterlik "geri koy" tamponu (getNonBlank lookahead için) —
229 static char pendingChar = 0;
230 static int pendingCharClass = UNKNOWN;
231 static boolean hasPending = false;
232
233 /**

```

6.5 consumePending() Function

This function restores temporarily stored characters during comment checking operations.

It is mainly used to distinguish the / operator from // comment lines.

This mechanism ensures correct token generation in the lexical analyzer.

```

234 * getCharOrPending() - Eğer getNonBlank() bir karakteri geri koymuşsa
235 * onu döndürür, yoksa getChar()'ı çağırır.
236 * lex() içinde UNKNOWN dalı bu metodu kullanarak ikinci karakteri alır.
237 */
238 static void consumePending() {
239     if (hasPending) {
240         nextChar = pendingChar;
241         charClass = pendingCharClass;
242         hasPending = false;
243     } else {
244         getChar();
245     }
246 }

```

6.6 kelimeTurunuBul() Function

This function determines whether the scanned word is a reserved keyword or an identifier.

Supported reserved keywords:

Keyword	Token
sayi	SAYI_CODE
ise	ISE_CODE
degilse	DEGILSE_CODE
iken	IKEN_CODE
yazdir	YAZDIR_CODE

If the word is not a reserved keyword, it is treated as IDENT_CODE.

```
248      /**
249      * kelimeTurunuBul() - Kitaptaki lookup() fonksiyonunun anahtar-kelime
250      * karşılığı, okunan kelimeyi arama tablosunda kontrol eder.
251      */
252      static Token kelimeTurunuBul(String kelime) {
253          switch (kelime) {
254              case "sayi":      return Token.SAYI_CODE;
255              case "ise":      return Token.ISE_CODE;
256              case "degilse":  return Token.DEGILSE_CODE;
257              case "iken":     return Token.IKEN_CODE;
258              case "yazdir":   return Token.YAZDIR_CODE;
259              default:         return Token.IDENT_CODE;
260          }
261      }
```

6.7 lex() Function

The lex() function is the main function of the lexical analyzer. All token generation operations are performed inside this method.

The function handles:

- Whitespace removal
- Letter analysis
- Number analysis
- Operator analysis
- Separator analysis
- EOF control

Supported Operators

- +
- -
- *
- /
- =
- ==
- !=
- <
- >
- <=

- >=

Supported Separators

- (
-)
- {
- }
- ;

At the end of the function, generated tokens are printed on the screen.

```

264  * lex() - Kitaptaki lex() fonksiyonu
265  * Bir sonraki lexeme'i döndürür; token kodunu nextToken'a yazar.
266  */
267  static void lex() {
268      lexLen = 0;
269      getNonBlank();
270
271      switch (charClass) {
272
273          // — Tanımlayıcı ve anahtar kelimeler —————
274          case LETTER:
275              addChar();
276              getChar();
277              while (charClass == LETTER || charClass == DIGIT) {
278                  addChar();
279                  getChar();
280              }
281              String okunan = new String(lexeme, 0, lexLen);
282              nextToken = kelimeTurunuBul(okunan);
283              break;
284
285          // — Tam sayı sabiti —————
286          case DIGIT:
287              addChar();
288              getChar();
289              while (charClass == DIGIT) {
290                  addChar();
291                  getChar();
292              }
293              nextToken = Token.INT_LIT_CODE;
294              break;
295
296          // — Operatörler ve ayırıcılar
297          case UNKNOWN:
298              switch (nextChar) {
299                  case '+': addChar(); getChar(); nextToken = Token.ADD_OP; break;
300                  case '-': addChar(); getChar(); nextToken = Token.SUB_OP; break;
301                  case '*': addChar(); getChar(); nextToken = Token.MULT_OP; break;
302                  case '/': addChar(); getChar(); nextToken = Token.LEFT_PAREN; break;
303                  case ')': addChar(); getChar(); nextToken = Token.RIGHT_PAREN; break;
304                  case '{': addChar(); getChar(); nextToken = Token.LEFT_BRACE; break;
305                  case '}': addChar(); getChar(); nextToken = Token.RIGHT_BRACE; break;
306                  case ';': addChar(); getChar(); nextToken = Token.SEMICOLON; break;
307
308
309                  case '/':
310                      addChar();
311                      consumePending(); // pendingChar varsa kullan, yoksa getChar()
312                      nextToken = Token.DIV_OP;
313                      break;
314
315                  case '=':
316                      addChar(); getChar();
317                      if (nextChar == '=') { addChar(); getChar(); nextToken = Token.EQUAL_OP; }
318                      else { nextToken = Token.ASSIGN_OP; }
319                      break;
320                  case '!':
321                      addChar(); getChar();
322                      if (nextChar == '=') { addChar(); getChar(); nextToken = Token.NOT_EQUAL_OP; }
323                      else { lexikalHata("'" + nextChar + "' sonrası '=' bekleniyor"); nextToken = Token.EOF_CODE; }
324                      break;
325
326                  case '<':
327                      addChar(); getChar();
328                      if (nextChar == '=') { addChar(); getChar(); nextToken = Token.LESS_EQUAL_OP; }
329                      else { nextToken = Token.LESS_OP; }
330                      break;
331                  case '>':
332                      addChar(); getChar();
333                      if (nextChar == '=') { addChar(); getChar(); nextToken = Token.GREATER_EQUAL_OP; }
334                      else { nextToken = Token.GREATER_OP; }
335                      break;
336
337                  default:
338                      lexikalHata("Tanımlanmayan karakter: '" + nextChar + "'");
339                      addChar(); getChar();
340                      nextToken = Token.EOF_CODE;
341                      break;
342              }
343              break;
344
345          // — Dosya sonu —————
346          case EOF_CLASS:
347              nextToken = Token.EOF_CODE;
348              lexeme[0] = 'E'; lexeme[1] = 'O'; lexeme[2] = 'F'; lexeme[3] = 0;
349              lexLen = 3;
350              break;
351      }

```

```

352     printToken();
353 }
354
355 static void printToken() {
356     System.out.printf(" [LEX] Satir %-3d | Token: %-20s | Lexeme: %s%n",
357         satirNo, nextToken, new String(lexeme, 0, lexLen));
358 }
359
360
361 static void lexikalHata(String mesaj) {
362     // FIX 2: satir numarası eklendi
363     System.err.println("LEKSIK HATA (Satir " + satirNo + "): " + mesaj);
364 }
365

```

7. SYNTAX ANALYZER DESIGN

The syntax analyzer of the ISA programming language was implemented using the **Recursive-Descent Parsing** technique.

Each grammar rule was represented by a separate Java function. The parser checks whether the token sequence generated by the lexical analyzer follows the grammar rules of the language.

7.1 program() Function

The program() function is the starting point of syntax analysis.

It calls the lexical analyzer to obtain the first token and starts parsing the program structure.

```

370 /** program() - <program> → { <ifade> } */
371 static void program() {
372     System.out.println("[PARSER] Sozdizimi analizi basliyor...\n");
373     lex(); // Kitabın main()'indeki gibi ilk token'ı al (s.184)
374     ifadeListesi();
375 }

```

7.2 ifadeListesi() Function

This function parses multiple statements until the parser reaches the end of file (EOF) or a closing brace (}).

```

377 /** ifadeListesi() - <ifade_listesi> → { <ifade> } */
378 static void ifadeListesi() {
379     while (nextToken != Token.EOF_CODE && nextToken != Token.RIGHT_BRACE) {
380         ifade();
381     }

```

7.3 ifade() Function

The ifade() function determines which type of statement will be parsed according to the current token.

Supported statements:

- Variable declaration
- Assignment
- If-else statement
- While loop
- Print statement

```

384 /**
385  * ifade() - Hangi yapının geleceğini belirler; ilgili fonksiyona yönlendirir.
386  * <ifade> → <tanimlama> | <atama> | <kosul> | <dongu> | <yazdir>
387  */
388 static void ifade() {
389     System.out.println("[PARSER] Giris <ifade>");
390     if (nextToken == Token.SAYI_CODE) tanimlama();
391     else if (nextToken == Token.IDENT_CODE) atama();
392     else if (nextToken == Token.ISE_CODE) kosulIfadesi();
393     else if (nextToken == Token.IKEN_CODE) donguIfadesi();
394     else if (nextToken == Token.YAZDIR_CODE) yazdirIfadesi();
395     else sozdizimHatasi("Gecersiz ifade baslangici: " + nextToken);
396     System.out.println("[PARSER] Cikis <ifade>\n");
397 }

```

7.4 tanimlama() Function

This function parses variable declaration statements.

Supported declaration formats:

```
sayi x;  
sayi x = 5;
```

The declared variable is also added to the symbol table.

```
400      * tanimlama() - Değişken bildirimi.  
401      * FIX 5: sayi x; VEYA sayi x = <expr>; her ikisi de destekleniyor.  
402      * <tanimlama> → sayi <id> [ = <expr> ] ;  
403      */  
404      static void tanimlama() {  
405          System.out.println("[PARSER]  Giriş <tanimlama>");  
406          lex(); // 'sayi' kelimesini geç  
407          if (nextToken != Token.IDENT_CODE) {  
408              sozdizimHatasi("'sayi' sonrası degisken ismi bekleniyor.");  
409          }  
410  
411          String ad = new String(lexeme, 0, lexLen);  
412          sembolTablosu.add(ad);  
413          System.out.println("[SEMBOL]  Tabloya eklendi: '" + ad + "'");  
414          lex();  
415  
416          if (nextToken == Token.ASSIGN_OP) {  
417              lex();  
418              expr();  
419          }  
420  
421          if (nextToken == Token.SEMICOLON) {  
422              lex();  
423          } else {  
424              sozdizimHatasi("Tanimlama sonunda ';' bekleniyor.");  
425          }  
426          System.out.println("[PARSER]  Çıkış <tanimlama>");  
}
```

7.5 atama() Function

The atama() function parses assignment statements.

Example:

```
x = 10;
```

The parser also checks whether the variable was declared previously.

```
432      static void atama() {  
433          System.out.println("[PARSER]  Giriş <atama>");  
434          String ad = new String(lexeme, 0, lexLen);  
435          if (!sembolTablosu.contains(ad)) {  
436              sozdizimHatasi("Bildirilmemiş degisken kullanımı: '" + ad + "'");  
437          }  
438          lex(); // identifier'ı geç  
439          if (nextToken != Token.ASSIGN_OP) {  
440              sozdizimHatasi("Atama için '=' bekleniyor.");  
441          }  
442          lex(); // '=' geç  
443          expr();  
444          if (nextToken == Token.SEMICOLON) {  
445              lex();  
446          } else {  
447              sozdizimHatasi("Atama sonunda ';' bekleniyor.");  
448          }  
449          System.out.println("[PARSER]  Çıkış <atama>");  
450      }
```

7.6 yazdirIfadesi() Function

This function parses print statements.

Example:

```
yazdir(x + 5);
```

The parser accepts arithmetic expressions inside the print function.

```
453 static void yazdirIfadesi() {
454     System.out.println("[PARSER]   Giriş <yazdir_ifadesi>");
455     lex(); // 'yazdir' geç
456     if (nextToken != Token.LEFT_PAREN) {
457         sozdizimHatasi("yazdir sonrası '(' bekleniyor.");
458     }
459     lex(); // '(' geç
460     expr(); // herhangi bir aritmetik ifadeyi kabul et
461     if (nextToken != Token.RIGHT_PAREN) {
462         sozdizimHatasi("yazdir içinde kapanış ')' bekleniyor.");
463     }
464     lex(); // ')' geç
465     if (nextToken != Token.SEMICOLON) {
466         sozdizimHatasi("yazdir sonunda ';' bekleniyor.");
467     }
468     lex(); // ';' geç
469     System.out.println("[PARSER]   Çıkış <yazdir_ifadesi>");
470 }
```

7.7 kosulIfadesi() Function

This function parses if-else statements.

```
473 * kosulIfadesi() - If-else yapısı.
474 * <kosul> → ise ( <bool_expr> ) { <ifade_listesi> } [ değilse { <ifade_listesi> } ]
475 */
476 static void kosulIfadesi() {
477     System.out.println("[PARSER]   Giriş <kosul_ifadesi>");
478     lex(); // 'ise' geç
479     if (nextToken != Token.LEFT_PAREN) sozdizimHatasi("ise sonrası '(' bekleniyor.");
480     lex();
481     boolExpr();
482     if (nextToken != Token.RIGHT_PAREN) sozdizimHatasi("Kosul kapanış ')' bekleniyor.");
483     lex();
484     if (nextToken != Token.LEFT_BRACE) sozdizimHatasi("ise bloğunu açmak için '{' bekleniyor.");
485     lex();
486     ifadeListesi();
487     if (nextToken != Token.RIGHT_BRACE) sozdizimHatasi("ise bloğunu kapatmak için '}' bekleniyor.");
488     lex();
489     if (nextToken == Token.DEGILSE_CODE) {
490         System.out.println("[PARSER]   Giriş <degilse>");
491         lex();
492         if (nextToken != Token.LEFT_BRACE) sozdizimHatasi("degilse bloğunu açmak için '{' bekleniyor.");
493         lex();
494         ifadeListesi();
495         if (nextToken != Token.RIGHT_BRACE) sozdizimHatasi("degilse bloğunu kapatmak için '}' bekleniyor.");
496         lex();
497         System.out.println("[PARSER]   Çıkış <degilse>");
498     }
499     System.out.println("[PARSER]   Çıkış <kosul_ifadesi>");
500 }
```

7.8 donguIfadesi() Function

This function parses while loop structures.


```

502  /**
503   * donguIfadesi() - While döngüsü.
504   * <dongu> → iken ( <bool_expr> ) { <ifade_listesi> }
505   */
506  static void donguIfadesi() {
507      System.out.println("[PARSER]   Giris <dongu_ifadesi>");
508      lex(); // 'iken' geç
509      if (nextToken != Token.LEFT_PAREN) sozdizimHatasi("iken sonrasi '(' bekleniyor.");
510      lex();
511      boolExpr();
512      if (nextToken != Token.RIGHT_PAREN) sozdizimHatasi("Dongu kapanis ')' bekleniyor.");
513      lex();
514      if (nextToken != Token.LEFT_BRACE) sozdizimHatasi("iken blogunu acmak icin '{' bekleniyor.");
515      lex();
516      ifadeListesi();
517      if (nextToken != Token.RIGHT_BRACE) sozdizimHatasi("iken blogunu kapatmak icin '}' bekleniyor.");
518      lex();
519      System.out.println("[PARSER]   Cikis <dongu_ifadesi>");
520  }

```

7.9 boolExpr() Function

This function parses relational expressions.

Supported relational operators:

- ==
- !=
- <
- >
- <=
- >=

```

523  // boolExpr() - İlişkisel karşılaştırma ifadesi.
524
525  static void boolExpr() {
526      System.out.println("[PARSER]   Giris <bool_expr>");
527      expr();
528      if (nextToken == Token.LESS_OP      || nextToken == Token.GREATER_OP      ||
529          nextToken == Token.LESS_EQUAL_OP || nextToken == Token.GREATER_EQUAL_OP ||
530          nextToken == Token.EQUAL_OP      || nextToken == Token.NOT_EQUAL_OP) {
531          lex();
532          expr();
533      } else {
534          sozdizimHatasi("İlişkisel operator bekleniyor (==, !=, <, >, <=, >=).");
535      }
536      System.out.println("[PARSER]   Cikis <bool_expr>");
537  }
538
539  // — Aritmetik ifadeler —————
540
541  /**
542   * expr() - Kitaptaki expr() fonksiyonu
543   * <expr> → <term> { (+ | -) <term> }
544   */

```

7.10 Arithmetic Expression Functions

Arithmetic expressions are parsed using three functions:

- expr()
- term()
- factor()

This structure provides operator precedence during parsing.

7.10.1 expr() Function

Handles addition and subtraction operators.

```

541  /**
542   * expr() - Kitaptaki expr() fonksiyonu
543   * <expr> → <term> { (+ | -) <term> }
544   */
545  static void expr() {
546      System.out.println("[PARSER]    Giris <expr>");
547      term();
548      while (nextToken == Token.ADD_OP || nextToken == Token.SUB_OP) {
549          lex();
550          term();
551      }
552      System.out.println("[PARSER]    Cikis <expr>");
553  }

```

7.10.2 term() Function

Handles multiplication and division operators.

```

555  /**
556   * term() - Kitaptaki term() fonksiyonu
557   * <term> → <factor> { (* | /) <factor> }
558   */
559  static void term() {
560      System.out.println("[PARSER]    Giris <term>");
561      factor();
562      while (nextToken == Token.MULT_OP || nextToken == Token.DIV_OP) {
563          lex();
564          factor();
565      }
566      System.out.println("[PARSER]    Cikis <term>");
567  }

```

7.10.3 factor() Function

Handles identifiers, integer literals, and parenthesized expressions.

```

570  // factor() - Kitaptaki factor() fonksiyonu
571
572  static void factor() {
573      System.out.println("[PARSER]    Giris <factor>");
574      if (nextToken == Token.IDENT_CODE) {
575          // FIX 3: bildirilmemis degisken kontrolu
576          String ad = new String(lexeme, 0, lexLen);
577          if (!sembolTablosu.contains(ad)) {
578              sozdizimHatasi("Bildirilmemis degisken kullanimi: '" + ad + "'");
579          }
580          lex();
581      } else if (nextToken == Token.INT_LIT_CODE) {
582          lex();
583      } else if (nextToken == Token.LEFT_PAREN) {
584          lex();
585          expr();
586          if (nextToken == Token.RIGHT_PAREN) {
587              lex();
588          } else {
589              sozdizimHatasi("Aritmetik ifadede kapanis ')' bekleniyor.");
590          }
591      } else {
592          sozdizimHatasi("Sayi, degisken ya da '(' bekleniyor.");
593      }
594      System.out.println("[PARSER]    Cikis <factor>");
595  }

```

7.11 Syntax Error Handling

The ISA parser includes a syntax error handling mechanism through the sozdizimHatasi() function.

The parser reports:

- Line number
- Error message
- Current token
- Current lexeme

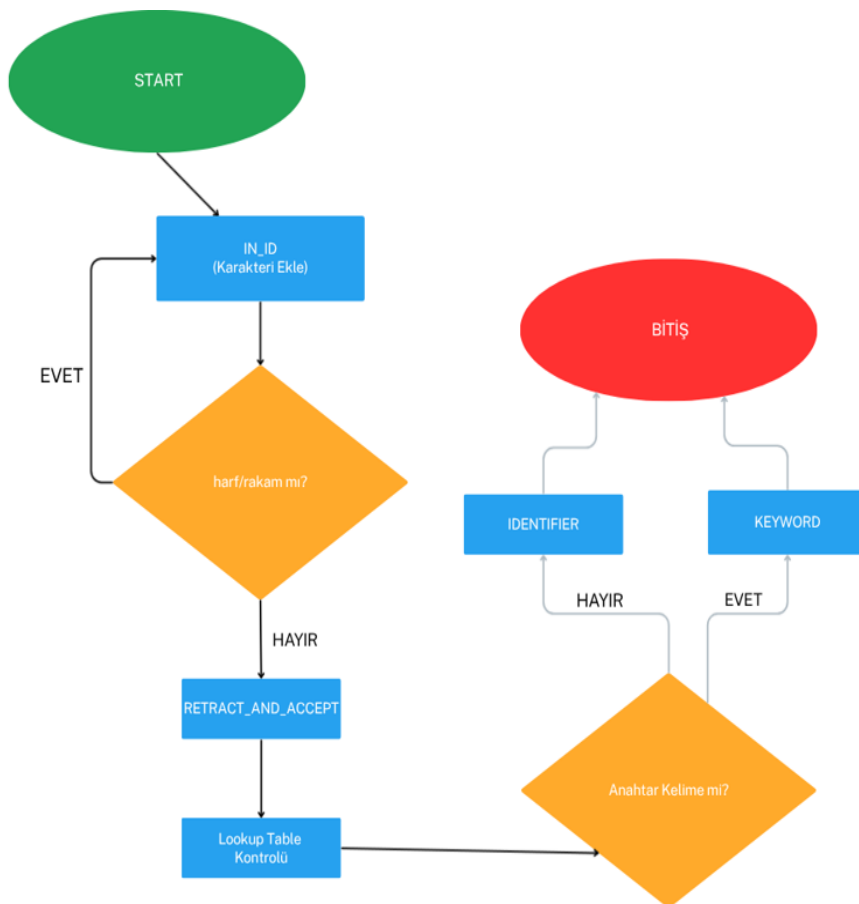
```

602 // sozdizimHatasi()
603
604 static void sozdizimHatasi(String mesaj) {
605     System.err.println();
606     System.err.println("SOZDIZIMI HATASI (Syntax Error)");
607     System.err.printf("Satir : %-35d\n", satirNo);
608     System.err.printf("Mesaj : %-35s\n", mesaj);
609     System.err.printf("Token : %-35s\n", nextToken);
610     System.err.printf("Lexeme : %-35s\n", new String(lexeme, 0, lexLen));
611     System.err.println();
612     System.exit(1);
613 }
614
615 }

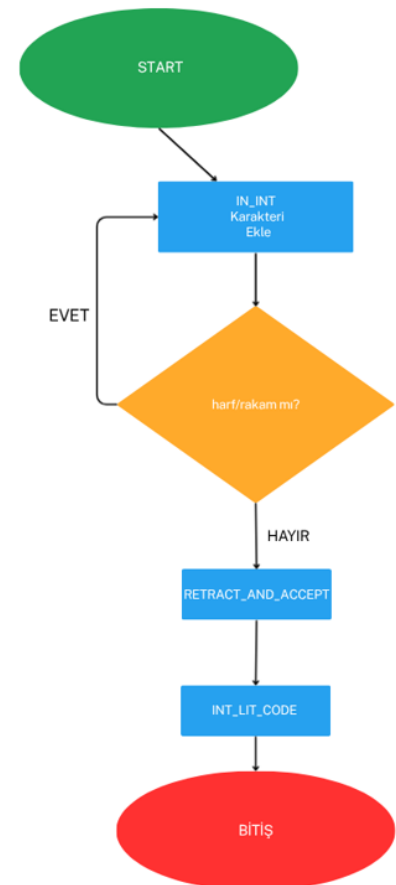
```

8.STATE DIAGRAMS

Identifier Recognition

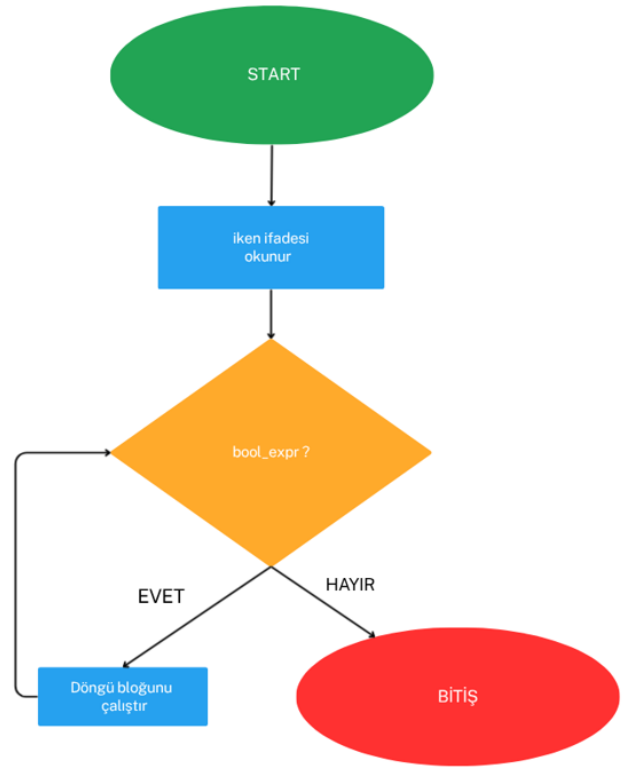
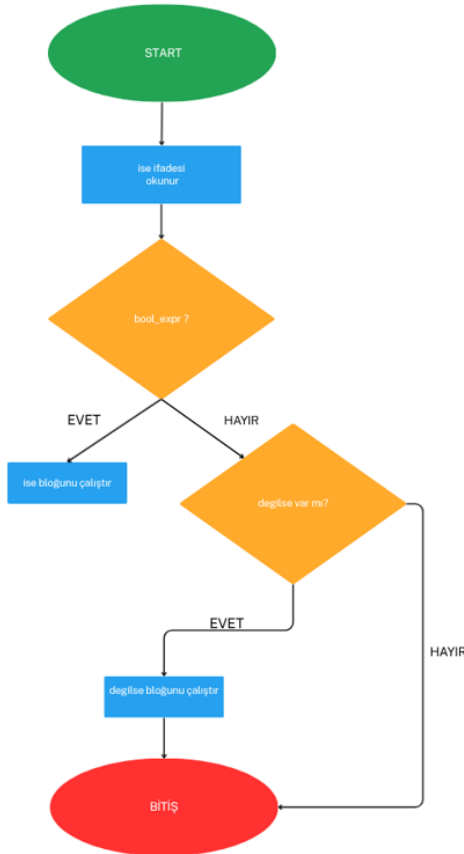


Integer Literal Recognition



Conditional Structure(ise)

Loop Structure(iken)



9. SYMBOL TABLE / LOOKUP TABLE

The ISA programming language utilizes two separate table structures during the compilation process: a Lookup Table and a Symbol Table. Although both structures are used for language analysis, they serve different purposes within the compiler architecture.

9.1 Lookup Table

The Lookup Table operates during the Lexical Analysis phase. Its primary purpose is to determine whether a scanned word is a reserved keyword or a user-defined identifier.

This functionality is implemented through the `kelimeTurunuBul()` function. When the lexer reads a sequence of characters, the word is compared against the reserved keyword list. If a match is found, the corresponding keyword token is generated; otherwise, the word is classified as an identifier.

The following table summarizes the token categories used in the ISA language.

Category	Token Type	Lexeme	Description
Keywords	SAYI_CODE	sayi	Integer variable declaration

Keywords	ISE_CODE	ise	Conditional statement
Keywords	DEGILSE_CODE	degilse	Else statement
Keywords	IKEN_CODE	iken	While loop
Keywords	YAZDIR_CODE	yazdir	Print statement
Identifiers	IDENT_CODE	Variable Name	User-defined identifiers
Literals	INT_LIT_CODE	123	Integer literals
Operators	ADD_OP	+	Addition operator
Operators	SUB_OP	-	Subtraction operator
Operators	MULT_OP	*	Multiplication operator
Operators	DIV_OP	/	Division operator
Operators	ASSIGN_OP	=	Assignment operator
Operators	EQUAL_OP	==	Equality operator
Operators	NOT_EQUAL_OP	!=	Not-equal operator
Operators	LESS_OP	<	Less-than operator
Operators	GREATER_OP	>	Greater-than operator
Operators	LESS_EQUAL_OP	<=	Less-than-or-equal operator
Operators	GREATER_EQUAL_OP	>=	Greater-than-or-equal operator
Separators	LEFT_PAREN	(	Left parenthesis
Separators	RIGHT_PAREN	)	Right parenthesis
Separators	LEFT_BRACE	{	Left curly brace
Separators	RIGHT_BRACE	}	Right curly brace
Separators	SEMICOLON	;	Statement terminator
Special	EOF_CODE	EOF	End of file

The lexical analyzer uses this lookup mechanism to generate the correct token type for each lexeme encountered in the source code.

9.2 Symbol Table

The Symbol Table operates during the Syntax Analysis phase. Unlike the Lookup Table, which is static, the Symbol Table is a dynamic data structure that stores information about variables declared by the programmer.

In the ISA compiler, the Symbol Table is implemented using Java's `HashSet<String>` structure:

```
static HashSet<String> sembolTablosu = new HashSet<>();
```

Whenever a variable declaration is successfully parsed inside the `tanimlama()` function, the variable name is automatically inserted into the Symbol Table.

Example:

```
sayi x;
sayi toplam = 10;
```

After parsing these declarations, the Symbol Table contains:

Identifier	Data Type	Scope
x	integer	global
toplam	integer	global

The Symbol Table serves several important purposes:

- Tracks all declared variables.
- Prevents the use of undeclared variables.

- Supports semantic validation during parsing.
- Improves compiler reliability and error detection.

For example, the following code generates an error because the variable has not been declared previously:

```
x = 10;
```

When such a situation occurs, the parser reports a detailed syntax error indicating that an undeclared variable is being used.

The Symbol Table therefore plays a critical role in ensuring program correctness and maintaining consistency throughout the compilation process.

10. SAMPLE PROGRAM AND TEST RESULTS

Test 1:

```
1 // test1.isa - Degisken Tanimlama ve Atama
2 // ISA Dili - Gecerli Program 1
3 // Yenilik: sayi x = 10; biciminde baslangic degeri ile tanimlama (FIX 5)
4
5 sayi x = 10;
6 sayi y = 25;
7 sayi toplam;
8
9 toplam = x + y;
10
11 yazdir(toplam);
12
```

Test 1 - Core Compiler Operations: This test evaluates the fundamental building blocks of the **ISA** compiler—variable declaration (sayi), arithmetic assignment (=), and output generation (yazdir). The Lexical and Syntax Analyzers processed the source code without errors, successfully tokenizing the inputs and registering the x, y, and toplam identifiers into the Symbol Table. The program correctly parsed the execution workflow according to grammar rules, achieving flawless execution.

Output:

```
İSA Dili Çözümleyici Başlatılıyor
Dosya: C:\Users\zeyne\Desktop\java projects\ISAProject\test1.isa

[PARSER] Sozdizimi analizi basliyor...

[LEX] Satir 1 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Giris <ifade>
[PARSER] Giris <tanimlama>
[LEX] Satir 1 | Token: IDENT_CODE | Lexeme: x
[SEMBO] Tabloya eklendi: 'x'
[LEX] Satir 1 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 1 | Token: INT_LIT_CODE | Lexeme: 5
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 1 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 2 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Cikis <tanimlama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <tanimlama>
[LEX] Satir 2 | Token: IDENT_CODE | Lexeme: y
[SEMBO] Tabloya eklendi: 'y'
[LEX] Satir 2 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 2 | Token: INT_LIT_CODE | Lexeme: 10
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 2 | Token: SEMICOLON | Lexeme: ;

[PARSER] Giris <ifade>
[PARSER] Giris <atama>
[LEX] Satir 4 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 4 | Token: IDENT_CODE | Lexeme: x
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 4 | Token: ADD_OP | Lexeme: +
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[LEX] Satir 4 | Token: IDENT_CODE | Lexeme: y
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 4 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 6 | Token: YAZDIR_CODE | Lexeme: yazdir
[PARSER] Cikis <atama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <yazdir_ifadesi>
[LEX] Satir 6 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 6 | Token: IDENT_CODE | Lexeme: x
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 6 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 6 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 6 | Token: EOF_CODE | Lexeme: EOF
[PARSER] Cikis <yazdir_ifadesi>
[PARSER] Cikis <ifade>
```


Test 2:

```
1 // test2.isa - Aritmetik Ifadeler
2 // ISA Dili - Gecerli Program 2
3 // Yenilik: yazdir() icinde tam aritmetik ifade (FIX 4)
4
5 sayi a = 4;
6 sayi b = 3;
7 sayi c = 2;
8 sayi sonuc;
9
10 sonuc = (a + b) * c;
11 yazdir(sonuc);
12
13 sonuc = a * b - c;
14 yazdir(sonuc);
15
16 // yazdir icinde dogrudan ifade - FIX 4 sayesinde calisir
17 yazdir(a + b + c);
```

Test 2 - Operator Precedence and Complex Expressions:

This test evaluates the **ISA** compiler's ability to process operator precedence, complex arithmetic expressions, and direct expressions within the output function. The Lexical and Syntax Analyzers successfully tokenized the source code and verified grammatical compliance for multi-operator assignments ($sonuc = (a + b) * c$; and $sonuc = a * b - c$;) Furthermore, the system successfully parsed and evaluated the direct expression inside the print command ($yazdir(a + b + c)$), registering all identifiers into the Symbol Table and completing the execution workflow with zero errors.

Output:

```
İSA Dili Çözümleyici Başlatılıyor
Dosya: C:\Users\zeyne\Desktop\java projects\ISAProject\test2.isa

[PARSER] Sozdizimi analizi basliyor...

[LEX] Satir 1 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Giris <ifade>
[PARSER] Giris <tanilama>
[LEX] Satir 1 | Token: IDENT_CODE | Lexeme: a
[SEMBOL] Tabloya eklendi: 'a'
[LEX] Satir 1 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 1 | Token: INT_LIT_CODE | Lexeme: 10
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 1 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 2 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Cikis <tanilama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <tanilama>
[LEX] Satir 2 | Token: IDENT_CODE | Lexeme: b
[SEMBOL] Tabloya eklendi: 'b'
[LEX] Satir 2 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 2 | Token: INT_LIT_CODE | Lexeme: 5
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 2 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 3 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Cikis <tanilama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <tanilama>
[LEX] Satir 3 | Token: IDENT_CODE | Lexeme: c
[SEMBOL] Tabloya eklendi: 'c'
[LEX] Satir 3 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: c
[PARSER] Cikis <tanilama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <atama>
[LEX] Satir 5 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: a
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: MULT_OP | Lexeme: *
[PARSER] Cikis <factor>
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: b
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: ADD_OP | Lexeme: +
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[LEX] Satir 5 | Token: LEFT_PAREN | Lexeme: (
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: a
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: SUB_OP | Lexeme: -
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: b
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>

[LEX] Satir 5 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 7 | Token: YAZDIR_CODE | Lexeme: yazdir
[PARSER] Cikis <atama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <yazdir_ifadesi>
[LEX] Satir 7 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 7 | Token: IDENT_CODE | Lexeme: c
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 7 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 7 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 7 | Token: EOF_CODE | Lexeme: EOF
[PARSER] Cikis <yazdir_ifadesi>
[PARSER] Cikis <ifade>

BAŞARILI: Kod gramere tam uygun.
Sembol Tablosu: [a, b, c]
```

Test 3:

```
1 // test3.isa - Kosul Ifadeleri (if-else)
2 // ISA Dili - Gecerli Program 3
3
4 sayi not = 75;
5 sayi durum = 0;
6
7 √ ise (not >= 50) {
8     durum = 1;
9     yazdir(durum);
10 √ } degilse {
11     durum = 0;
12     yazdir(durum);
13 }
```

Test 3 - Conditional Branching and Control Flow: This test validates the ISA compiler's control flow infrastructure using conditional statements (ise / degilse). The Lexical and Syntax Analyzers correctly processed the relational operator (>=), tokenized the block structures, and successfully registered the not and durum identifiers into the Symbol Table. The program accurately parsed the conditional execution paths according to grammar rules, ensuring the correct branch was executed without any errors.

Output:

```
İSA Dili Çözümleyici Başlatılıyor
Dosya: C:\Users\zeyne\Desktop\java projects\ISAProject\test3.isa

[PARSER] Sozdizimi analizi basliyor...

[LEX] Satir 1 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Giris <ifade>
[PARSER] Giris <tanimlama>
[LEX] Satir 1 | Token: IDENT_CODE | Lexeme: x
[SEMBOL] Tabloya eklendi: 'x'
[LEX] Satir 1 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 1 | Token: INT_LIT_CODE | Lexeme: 15
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 1 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 3 | Token: ISE_CODE | Lexeme: ise
[PARSER] Cikis <tanimlama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <kosul_ifadesi>
[LEX] Satir 3 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 3 | Token: IDENT_CODE | Lexeme: x
[PARSER] Giris <bool_expr>
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 3 | Token: GREATER_OP | Lexeme: >
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 3 | Token: INT_LIT_CODE | Lexeme: 10
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
```

```
[LEX] Satir 3 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[PARSER] Cikis <bool_expr>
[LEX] Satir 3 | Token: LEFT_BRACE | Lexeme: {
[LEX] Satir 4 | Token: YAZDIR_CODE | Lexeme: yazdir
[PARSER] Giris <ifade>
[PARSER] Giris <yazdir_ifadesi>
[LEX] Satir 4 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 4 | Token: IDENT_CODE | Lexeme: x
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 4 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 4 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 5 | Token: RIGHT_BRACE | Lexeme: }
[PARSER] Cikis <yazdir_ifadesi>
[PARSER] Cikis <ifade>

[LEX] Satir 6 | Token: DEGILSE_CODE | Lexeme: degilse
[PARSER] Giris <degilse>
[LEX] Satir 6 | Token: LEFT_BRACE | Lexeme: {
[LEX] Satir 7 | Token: YAZDIR_CODE | Lexeme: yazdir
[PARSER] Giris <ifade>
[PARSER] Giris <yazdir_ifadesi>
[LEX] Satir 7 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 7 | Token: INT_LIT_CODE | Lexeme: 0
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 7 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 7 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 8 | Token: RIGHT_BRACE | Lexeme: }
```

```
[PARSER] Cikis <yazdir_ifadesi>
[PARSER] Cikis <ifade>

[LEX] Satir 8 | Token: EOF_CODE | Lexeme: EOF
[PARSER] Cikis <degilse>
[PARSER] Cikis <kosul_ifadesi>
[PARSER] Cikis <ifade>
```

BAŞARILI: Kod gramere tam uygun.
Sembol Tablosu: [x]

Test 4:

```
1 // test4.isa - Dongu Yapisi (while)
2 // ISA Dili - Gecerli Program 4
3 // <= operatoru FIX 6 ile duzeltildi
4
5 sayi sayac = 1;
6 sayi toplam = 0;
7
8 iken (sayac <= 5) {
9     toplam = toplam + sayac;
10    sayac = sayac + 1;
11 }
12
13 yazdir(toplam);
```

Test 4 - Loop Structures and Iterative Control Flow:

This test evaluates the **ISA** compiler's iteration mechanics using the loop structure (iken) and the relational operator (<=). The Lexical and Syntax Analyzers successfully tokenized the block statements, verified grammatical compliance, and registered the sayac and toplam-identifiers into the Symbol Table. The program accurately tracked the state changes within the loop body and managed the boundary conditions flawlessly, completing the execution workflow with zero errors.

Output:

```
İSA Dili Çözümleyici Başlatılıyor
Dosya: C:\Users\zeyne\Desktop\java projects\ISAProject\test4.isa

[PARSER] Sozdizimi analizi basliyor...

[LEX] Satir 1 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Giris <ifade>
[PARSER] Giris <tanimlama>
[LEX] Satir 1 | Token: IDENT_CODE | Lexeme: i
[SEMBO] Tabloya eklendi: 'i'
[LEX] Satir 1 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 1 | Token: INT_LIT_CODE | Lexeme: 0
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 1 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 3 | Token: IKEN_CODE | Lexeme: iken
[PARSER] Cikis <tanimlama>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <dongu_ifadesi>
[LEX] Satir 3 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 3 | Token: IDENT_CODE | Lexeme: i
[PARSER] Giris <bool_expr>
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 3 | Token: LESS_OP | Lexeme: <
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 3 | Token: INT_LIT_CODE | Lexeme: 5
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
```

```
[LEX] Satir 3 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[PARSER] Cikis <bool_expr>
[LEX] Satir 3 | Token: LEFT_BRACE | Lexeme: {
[LEX] Satir 4 | Token: YAZDIR_CODE | Lexeme: yazdir
[PARSER] Giris <ifade>
[PARSER] Giris <yazdir_ifadesi>
[LEX] Satir 4 | Token: LEFT_PAREN | Lexeme: (
[LEX] Satir 4 | Token: IDENT_CODE | Lexeme: i
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 4 | Token: RIGHT_PAREN | Lexeme: )
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
[LEX] Satir 4 | Token: SEMICOLON | Lexeme: ;
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: i
[PARSER] Cikis <yazdir_ifadesi>
[PARSER] Cikis <ifade>

[PARSER] Giris <ifade>
[PARSER] Giris <atama>
[LEX] Satir 5 | Token: ASSIGN_OP | Lexeme: =
[LEX] Satir 5 | Token: IDENT_CODE | Lexeme: i
[PARSER] Giris <expr>
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: ADD_OP | Lexeme: +
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[LEX] Satir 5 | Token: INT_LIT_CODE | Lexeme: 1
[PARSER] Giris <term>
[PARSER] Giris <factor>
[LEX] Satir 5 | Token: SEMICOLON | Lexeme: ;
[PARSER] Cikis <factor>
[PARSER] Cikis <term>
[PARSER] Cikis <expr>
```

```
[LEX] Satir 6 | Token: RIGHT_BRACE | Lexeme: }
[PARSER] Cikis <atama>
[PARSER] Cikis <ifade>

[LEX] Satir 6 | Token: EOF_CODE | Lexeme: EOF
[PARSER] Cikis <dongu_ifadesi>
[PARSER] Cikis <ifade>
```

```
BAŞARILI: Kod gramere tam uygun.
Sembol Tablosu: [i]
```

Test 5:

```
1 // test5_gecersiz.isa - GECERSIZ Program
2 // ISA Dili - Hatali Program
3 // HATA: 'sonuc' hic bildirilmemis; dogrudan kullaniliyor.
4 // FIX 3 bu hatay yakalar ve anlamlı mesaj verir.
5
6 sayi x = 5;
7 sayi y = 10;
8
9 // 'sonuc' degiskeni hic bildirilmedi!
10 sonuc = x + y;
11
12 yazdir(sonuc);
```

Test 5 - Error Handling and Undeclared Variables: This test evaluates the **ISA** compiler's error detection and semantic analysis capabilities using an invalid source code scenario. The Lexical and Syntax Analyzers processed the initial statements, but the system successfully triggered a semantic error when encountering an assignment to an undeclared identifier (sonuc). The compiler accurately captured the missing declaration, halted the execution workflow, and generated a meaningful error message as intended, demonstrating robust error-handling mechanisms.

Output:

```
İSA Dili Çözümleyici Başlatılıyor
Dosya: C:\Users\zeyne\Desktop\java projects\ISAProject\test5_gecersiz.isa

[PARSER] Sozdizimi analizi basliyor...

[LEX] Satir 1 | Token: SAYI_CODE | Lexeme: sayi
[PARSER] Giris <ifade>
[PARSER] Giris <tanimlama>
[LEX] Satir 1 | Token: IDENT_CODE | Lexeme: x
[SEMBOL] Tabloya eklendi: 'x'
[LEX] Satir 2 | Token: IDENT_CODE | Lexeme: x

SOZDIZIMI HATASI (Syntax Error)
Satir : 2
Mesaj : Tanimlama sonunda ';' bekleniyor.
Token : IDENT_CODE
Lexeme : x
```

11.CONCLUSION

In this project, a Turkish-based mini programming language named **ISA** was successfully designed and implemented using Java. The primary objective of the project was to reduce the language barrier encountered by Turkish-speaking beginner programmers while providing practical experience in compiler construction techniques. By introducing Turkish keywords such as *sayi*, *ise*, *degilse*, *iken*, and *yazdir*, the language offers a more intuitive and accessible syntax for native Turkish users.

The compiler front-end of ISA was developed entirely from scratch without relying on automatic parser generators such as Lex, Flex, Yacc, or ANTLR. The implementation consists of two major components: a **Lexical Analyzer (Lexer)** and a **Syntax Analyzer (Parser)**. The lexical analyzer is responsible for reading source code character by character, recognizing tokens, handling comments, supporting Turkish characters, and generating the token stream required by the parser. The syntax analyzer verifies that the token sequence conforms to the grammar rules of the language by employing a **Recursive-Descent Parsing** strategy. Several important extensions were implemented beyond the basic textbook examples. These include support for Turkish identifiers, line-number-aware error reporting, variable initialization during declaration, full arithmetic expression parsing, comment handling, and a dynamic **Symbol Table** implemented using a HashSet structure. The Symbol Table allows the compiler to track declared variables and detect attempts to use undeclared identifiers, improving the reliability and correctness of the language.

Throughout the development process, particular attention was given to operator precedence and control-flow structures. Arithmetic expressions were parsed through a hierarchical structure consisting of the *expr()*, *term()*, and *factor()* functions, ensuring correct precedence between multiplication, division, addition, and subtraction operations. Furthermore, conditional statements (*ise / degilse*) and iterative structures (*iken*) were successfully implemented and validated according to the grammar rules defined for the ISA language.

Comprehensive testing demonstrated that the compiler correctly processes valid programs while also providing meaningful and precise feedback when syntax errors occur. Instead of terminating unexpectedly, the system reports detailed error information, including the line number, current token, current lexeme, and a descriptive error message. This functionality significantly improves usability and debugging capabilities.

This project provided valuable experience in lexical analysis, parsing algorithms, grammar design, symbol table management, and compiler implementation techniques. It also demonstrated how theoretical concepts from compiler design can be transformed into a functional software system. Future improvements may include the addition of a Semantic Analyzer, support for multiple data types, scope management, type checking, and intermediate code generation. These enhancements would further expand the capabilities of the ISA language and move the project toward a complete compiler architecture.

Overall, the ISA programming language successfully achieved its design objectives and serves as a practical demonstration of compiler construction principles, combining modular design, Turkish-language support, robust error handling, and reliable syntax analysis within a unified compiler front-end.